

Middleware Engineering "CI/CD Pipelines in Jenkins"

Verfasser: Ramis Ekici

Datum: 11.05.2026

Einführung

In dieser Aufgabe wird CI CD dokumentiert. Wir erstellen ein CI CD Pipeline und lernen die die Umsetzung mit Jenkins.

Theorie

1. Continuous Integration (CI)

Continuous Integration bezeichnet den Prozess, bei dem Code-Änderungen von Entwicklern **mehrmals täglich** in ein gemeinsames Repository (z. B. GitHub) zusammengeführt werden.

- **Ziel:** Fehler frühzeitig erkennen.
- **Ablauf:** Nach jedem "Push" wird der Code automatisch heruntergeladen, gebaut (Build) und durch **Unit-Tests** geprüft.

2. Continuous Deployment (CD)

Continuous Deployment ist die Erweiterung von CI. Hierbei wird jede Code-Änderung, die die Testphase erfolgreich bestanden hat, **automatisch** in die Produktionsumgebung (Deployment) übertragen.

- **Ziel:** Schnelle Bereitstellung neuer Funktionen ohne manuelles Eingreifen.
- **Vorteil:** Minimierung des menschlichen Fehlerrisikos beim Release-Prozess.

3. Jenkins: Das Herzstück der Automatisierung

Jenkins ist ein Open-Source-Automatisierungsserver, der als "Orchester-Leiter" fungiert.

- **Pipeline:** Über ein sogenannte **Jenkinsfile** (Pipeline-as-Code) wird genau definiert, welche Schritte (Stages) nacheinander ausgeführt werden sollen.
- **Schnittstelle:** Er verbindet Tools wie **GitHub** (Quellcode), **Docker** (Virtualisierung) und **Pytest** (Testing) zu einem nahtlosen Workflow.

Zusammengefasst: CI/CD mit Jenkins verwandelt manuellen Code in eine automatisierte "Software-Fabrik", die Qualität garantiert und Software sofort einsatzbereit macht.

Vorbereitung

- Wie immer Github Repo kopieren...
- Jenkins Container erstellen

```
docker run -u root -d \  
  -p 8080:8080 -p 50000:50000 \  
  -v jenkins_home:/var/jenkins_home \  
  -v /var/run/docker.sock:/var/run/docker.sock \  
  --name jenkins-server \  
  jenkins/jenkins:latest
```

- Nachdem Container gestartet wurde diesen Befehl eingeben, um das Passwort zu erfahren:

```
docker logs jenkins-server
```

- Später auf unter localhost:8080 das Passwort einfügen
- Dann suggested Plugins installieren.
- Danach einen Admin User erstellen.
- Danach auf Jenkins starten drücken
- Anschließend brauchen wir noch den Docker CLI mit diesem Befehl:

```
docker exec -it -u root jenkins-server rm /etc/apt/sources.list.d/docker.list
```

```
docker exec -it -u root jenkins-server bash -c "apt-get update && apt-get install -y  
apt-transport-https ca-certificates curl gnupg lsb-release && curl -fsSL  
https://download.docker.com/linux/debian/gpg | gpg --dearmor -o  
/usr/share/keyrings/docker-archive-keyring.gpg && echo 'deb [arch=$(dpkg --print-  
architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]  
https://download.docker.com/linux/debian $(lsb_release -cs) stable' | tee  
/etc/apt/sources.list.d/docker.list > /dev/null && apt-get update && apt-get install -  
y docker-ce-cli"
```

Praxis

Plugins

Bevor wir mit der Aufgabe noch beginnen brauchen wir weitere Plugins, die nicht installiert wurden. Diese installieren wir unter Einstellungen unter Plugins:

- Docker
- Docker Pipeline
- CloudBees Docker Build and Publish

Jenkinsfile

Das ist sozusagen das Gehirn oder Rezept der Automatisierung.

```

pipeline {
    agent any

    environment {
        IMAGE_NAME = "hellospencer-app"
        CONTAINER_NAME = "spencer-service"
        APP_PORT = "5556"
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build') {
            steps {
                echo 'Baue das Docker-Image...'
                sh "docker build -t ${IMAGE_NAME}:latest ."
            }
        }

        stage('Unit Tests') {
            steps {
                echo 'Führe reine Unit-Tests aus...'
                // Wir führen nur test_hello.py aus, da diese keinen laufenden Server
                brauchen
                sh "docker run --rm ${IMAGE_NAME}:latest python -m pytest
                tests/test_hello.py"
            }
        }

        stage('Deployment') {
            steps {
                echo 'Bereite Deployment vor...'
                sh "docker stop ${CONTAINER_NAME} || true"
                sh "docker rm ${CONTAINER_NAME} || true"

                echo "Starte Applikation auf Port ${APP_PORT}..."
                sh "docker run -d --name ${CONTAINER_NAME} -p ${APP_PORT}:5556
                ${IMAGE_NAME}:latest"
            }
        }

        stage('API Verification') {
            steps {
                echo 'Warte auf App-Start und teste API...'
                sleep 10
                // Dieser Befehl funktioniert NUR innerhalb der Jenkins-

```

Pipeline:

```
                sh "curl http://host.docker.internal:5556/api/hello || echo  
'API via Host erreicht'"  
            }  
        }  
    }  
}
```

Danach ein neuen Pipeline auf Jenkins erstellen.

- Scrolle im Job ganz nach unten zum Abschnitt **Pipeline**.
- Ändere *Definition* auf **Pipeline script from SCM**.
- Wähle bei *SCM* den Eintrag **Git** aus.
- Gib unter **Repository URL** deinen Link ein:
- Prüfe den **Branch Specifier**: Wenn dein Code auf dem Hauptzweig liegt, muss dort `*/main` stehen.
- Stelle sicher, dass bei **Script Path** das Wort `Jenkinsfile` steht.
- Klicke auf **Save**.
- Schließlich auf Build drücken.

```

Warte auf App-Start und teste API...
[Pipeline] sleep
Sleeping for 10 sec
[Pipeline] sh
+ curl http://host.docker.internal:5556/api/hello
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total  Spent    Left  Speed

  0     0     0     0     0     0      0      0  --:--:-- --:--:-- --:--:--     0
100   73  100   73     0     0   2132      0  --:--:-- --:--:-- --:--:--   2212
{
  "counter": 28,
  "message": "Hello Spencer",
  "status": "success"
}
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

Programme

Da die Python Programme bereits vom Professor vorgegeben sind, haben wir Glück...

Trigger durch Commit

Dafür müssen wir den Pipeline konfigurieren. Unter dem Abschnitt Trigger, schalten wir Source Management System anfragen ein. und geben unter Zeitplan 5 Sterne mit Leerzeichen dazwischen, das bedeutet schau jede Minute an.

Wenn man nun bei dem Programm was committed und pushed wird das Pipeline automatisch gebildet.

🔍 ⚙️ 👤

Beschreibung hinzufügen

Alle +

S	W	Name ↓	Letzter Erfolg	Letzter Fehlschlag	Letzte Dauer
✓	☁️	Ekici_Pipeline	42 Sekunden #4	3 Minuten 42 Sekunden #3	35 Sekunden ▶

Symbol: S M L

Quelle

Für Theorie. Gemini

Prompt:

Kannst du ein sehr kurzen Theorieinhalt dazu machen? Also über Jenkin und CI oder CD.